

Relatório Técnico

1. Objetivo do Case

Como os dados das redes sociais podem contribuir para definir e melhorar o posicionamento do Santander perante aos seus clientes e potenciais clientes

- Obter dados que citem o Santander com base nas postagens das redes sociais
- Aproveitar a visibilidade do nome Santander através de sua participação ativa como patrocinador de eventos culturais como o “Todo mundo no Rio 2025” que traz a cantora Lady Gaga para um show gratuito na praia de Copacabana
- Modelar, juntamente com dados já existentes, a forma que as postagens (que funcionam como publicidade indireta) podem afetar positivamente ou negativamente o Santander
- Com base nesses dados, podemos perceber clientes que só se tornaram clientes por causa do eventos, clientes antigos que podem ter se decepcionado e tem chance de deixar o banco.
- Com base nisso, é possível até mesmo prever a possibilidade de perda financeira da instituição com clientes abandonando a instituição
- Outro fator que pode impulsionar a importância do case: o banco investe muito nos dados do OBK. Com as impressões de redes sociais, novos dados serão disponibilizados e conheceremos melhor onde o Santander está posicionado em relação ao pensamento de seus clientes e não clientes

O objetivo do case é desenvolver uma solução de engenharia de dados para consumir informações das APIs de rede sociais em tempo real, porém foi utilizado a biblioteca Faker para simular os dados das APIs. O fluxo de dados deve ser processado e armazenado em um Data Lake, estruturado em camadas (bronze, silver, gold). Além disso, a solução deve permitir monitoramento e observabilidade, garantindo a rastreabilidade do fluxo de dados, detecção de falhas e identificação de gargalos de desempenho.

Github: <https://github.com/diegomeyer/datamaster>

2. Arquitetura de Solução e Arquitetura Técnica

2.1. Arquitetura de Solução

A solução foi projetada em um pipeline de dados com as seguintes etapas principais:

- Extração:
 - i. Geração dos dados via biblioteca Faker
 - ii. As informações são enviadas para o tópico no Kafka.
- Processamento em Tempo Real:
 - i. Dados do Kafka são consumidos pelo Spark.
 - ii. Os dados são escritos na camada bronze do Data Lake.

- Transformação e Limpeza:
 - i. Dados da camada bronze são processados e normalizados para a camada silver.
- Agregação:
 - i. A camada gold é criada com agregações e métricas prontas para análise, como KPIs.
- Monitoramento:
 - i. Métricas de todo o pipeline são monitorados usando Prometheus e Grafana.

2.2. Arquitetura Técnica

Tecnologias Utilizadas:

Tecnologia	Função
Kafka	Sistema de mensageria para ingestão de dados em tempo real.
Kafka Exporter	Export as métricas do Kafka para o Prometheus
Spark	Processamento distribuído de dados.
HDFS	Armazenamento em Data Lake com suporte a grandes volumes.
Jupyter	Ambiente para exploração dos dados
Airflow	Scheduler Jobs
Prometheus	Coleta de métricas para monitoramento.
Grafana	Visualização de métricas em dashboards.
Docker Compose	Orquestração dos serviços.

Arquitetura:



3. Explicação sobre o Case Desenvolvido

3.1. Extração de Dados/Geração de dados Fake.

O script `fake_api.py` gera os dados para cada rede social (instagram, facebook, x)

3.2. Processamento de Dados

O script `kafka_to_bronze.py` de cada app (instagram_ingestion, facebook_ingestion, x_ingestion):

- Consome mensagens do Kafka em tempo real no tópico match.
- Escreve os dados brutos e as métricas de cada etapa na camada bronze do Data Lake em formato Parquet.
- Assegura tolerância a falhas com checkpoints.

```

--
print(spark.read.parquet("hdfs://hadoop-namenode:8020/dataLake/bronze/instagram/*.parquet").printSchema())
except AnalysisException as e:
    print("Erro do Spark ao acessar os dados: (e)")

root
 |-- id: string (nullable = true)
 |-- user handle: string (nullable = true)
 |-- caption: string (nullable = true)
 |-- image url: string (nullable = true)
 |-- posted at: string (nullable = true)
 |-- likes: integer (nullable = true)
 |-- hashtags: array (nullable = true)
 |   |-- element: string (containsNull = true)
 |-- comments: array (nullable = true)
 |   |-- element: struct (containsNull = true)
 |       |-- user: string (nullable = true)
 |       |-- comment: string (nullable = true)
 |       |-- timestamp: string (nullable = true)
 |-- event_time: timestamp (nullable = true)

None

: try:
    print(spark.read.parquet("hdfs://hadoop-namenode:8020/dataLake/bronze/facebook/*.parquet").printSchema())
except AnalysisException as e:
    print("Erro do Spark ao acessar os dados: (e)")

root
 |-- id: string (nullable = true)
 |-- user name: string (nullable = true)
 |-- post content: string (nullable = true)
 |-- created at: string (nullable = true)
 |-- likes: integer (nullable = true)
 |-- shares: integer (nullable = true)
 |-- comments: array (nullable = true)
 |   |-- element: struct (containsNull = true)
 |       |-- user: string (nullable = true)
 |       |-- comment: string (nullable = true)
 |       |-- timestamp: string (nullable = true)
 |-- event_time: timestamp (nullable = true)

None

: try:
    print(spark.read.parquet("hdfs://hadoop-namenode:8020/dataLake/bronze/x/*.parquet").printSchema())
except AnalysisException as e:
    print("Erro do Spark ao acessar os dados: (e)")

root
 |-- username: string (nullable = true)
 |-- tweet: string (nullable = true)
 |-- created at: string (nullable = true)
 |-- likes: integer (nullable = true)
 |-- retweets: integer (nullable = true)
 |-- verified: boolean (nullable = true)
 |-- replies: array (nullable = true)
 |   |-- element: struct (containsNull = true)
 |       |-- username: string (nullable = true)
 |       |-- tweet: string (nullable = true)
 |       |-- created_at: string (nullable = true)
 |-- event_time: timestamp (nullable = true)

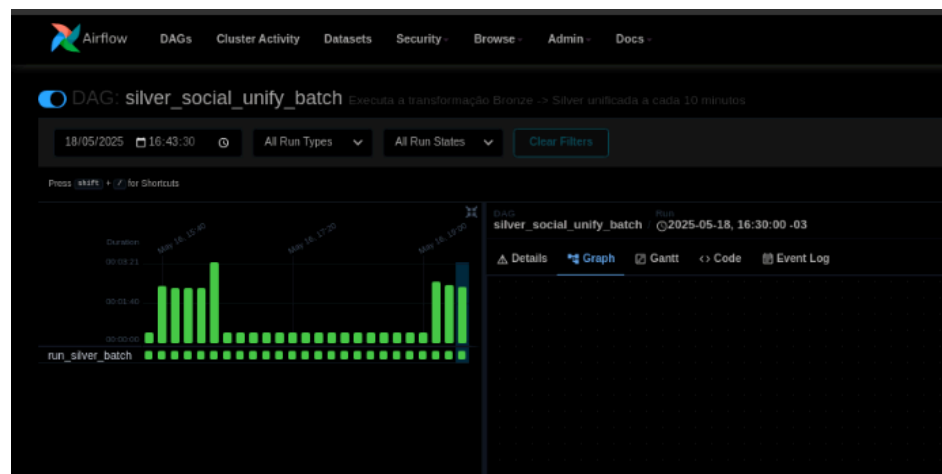
None

```

Schema - Tabela Bronze

O script `silver_unified_batch.py`

- DAG que consome a camada bronze e unifica as 3 tabelas bronze.



DAG - Bronze to Silver

```
[10]: spark.read.parquet("hdfs://hadoop-namenode:8020/datalake/silver/social_media/*.parquet").count()

[10]: 38154

[11]: spark.read.parquet("hdfs://hadoop-namenode:8020/datalake/silver/social_media/*.parquet").show(100)
```

author	content	post_date	likes	comments	shares	source
Elizabeth Reese	Art hope pattern ...	2025-05-10 11:29:...	1802	{{Dr. Mia Myers, ...	44	facebook
Amanda Welch	Coach relate out ...	2025-05-09 12:02:...	1737	{{Lisa Torres, Ex...	92	facebook
Gabriel Mills	Kid investment be...	2025-05-15 13:16:...	4959	{{Anthony Jensen,...	177	facebook
Mrs. Nicole Horne	Computer relation...	2025-04-29 23:32:...	1733	{{Janet Nguyen, W...	167	facebook
Natalie Good	Part chair series...	2025-04-17 16:26:...	4960	{{Jennifer Pollar...	185	facebook
Adrian Miller	Candidate have ge...	2025-04-28 07:01:...	4852	{{Zachary Whitake...	186	facebook
Amanda Stewart	Loss far material...	2025-04-25 03:28:...	453	{{James Charles, ...	153	facebook
Michael Torres	Clear coach commu...	2025-05-11 23:44:...	2408	{{Rita Roberts, O...	110	facebook
Martha Smith	Give into court. ...	2025-05-11 06:48:...	533	{{Robert Guerrero...	38	facebook
Emily Cohen	Wall too necessar...	2025-05-01 19:54:...	288	{{Brandi Gonzalez...	82	facebook
Michael Lynch	Ever when away no...	2025-05-13 12:06:...	4234	{{Michael Rogers...	12	facebook
Angel Smith	Brother onto degr...	2025-04-29 08:32:...	4890	{{Richard Reed, F...	165	facebook
Justin Rodriguez	The address usual...	2025-04-24 22:18:...	3570	{{Thomas Rogers, ...	195	facebook
Mark Brown	Sing beyond state...	2025-04-20 16:58:...	847	{{Stephanie Lee, ...	182	facebook
Candice Moyer	Might perform net...	2025-04-22 09:04:...	4704	{{Ann Duncan, Com...	163	facebook

Saídas das tabelas Silvers

O script gold_batch.py

- DAG que consome a camada silver e realiza agregações.



DAG - Silver To Gold

```
[12]: spark.read.parquet("hdfs://hadoop-namenode:8020/datalake/gold/social_media/engajamento_diario/*/*.parquet").show(5)
```

post_day	total_posts	total_likes	total_shares	total_comments
2025-05-08	316	1583717	805852	1497
2025-04-27	389	1513484	730855	1568
2025-04-26	329	1623276	836220	1672
2025-05-06	386	1538256	785163	1485
2025-04-15	144	729813	359128	706

only showing top 5 rows

```
[13]: spark.read.parquet("hdfs://hadoop-namenode:8020/datalake/gold/social_media/top_autores/*/*.parquet").show(5)
```

author	post_count	likes_total	shares_total	comments_total	total_engajamento
ubryant	5	42065	14034	19	56118
vjohnson	4	32584	12159	10	44673
xsmith	7	26216	16261	30	42507
chris04	3	27867	13881	24	41772
andrew53	3	27420	13629	27	41076

only showing top 5 rows

```
[14]: spark.read.parquet("hdfs://hadoop-namenode:8020/datalake/gold/social_media/posts_por_hora/*/*.parquet").show(5)
```

hour	total_posts
0	459
1	428
2	418
3	396
4	381

only showing top 5 rows

Saídas das tabelas Gold

3.3. Estrutura do Data Lake

- Camada Bronze: Dados brutos conforme recebidos do Kafka.
- Camada Silver: Dados transformados e normalizado.
- Camada Gold: Dados agregados, como KPI.

3.4. Monitoramento

Métricas:

- Kafka, HDFS e Spark expõem métricas para o Prometheus, que são visualizadas no Grafana.



Hadoop Métricas



Kafka Métricas

3.5. LGPD

Para os dados que trabalhamos na API, utilizamos técnica de hashing (SHA-256 com salt) para anonimizar o campo **author** na camada Silver

Vantagens:

- **Preserva a capacidade de análise de comportamento do usuário:** Rastrear a atividade do mesmo usuário ao longo do tempo e em diferentes partes da sua análise.
- **Permite a criação de features agregadas por usuário:** Contagem de posts, frequência de interação, etc.
- **Anonimização robusta:** O **username** original é perdido, reduzindo significativamente o risco de identificação direta.
- **Implementação relativamente simples** com as funções do PySpark.

Desvantagens:

- **Perda da capacidade de identificar usuários específicos:** Se em algum momento for necessário correlacionar a análise com a identidade real dos usuários (por exemplo, para responder a uma solicitação da LGPD de acesso aos dados), isso não será possível diretamente.

4. Melhorias

4.1. Escalabilidade:

- Implementar particionamento no Kafka e paralelismo no Spark para suportar maior volume de dados.
- Configurar múltiplos nós no cluster HDFS para maior capacidade de armazenamento.

4.2. Observabilidade Avançada:

- Adicionar rastreamento distribuído com OpenTelemetry para monitorar o tempo de processamento em cada componente do pipeline.

4.3. Governança de Dados:

- Adicionar políticas de retenção em cada camada.
- Adicionar expurgo dos dados em todas as camadas não apenas na bronze

4.4. Segurança:

- Configurar autenticação e autorização no Kafka.
- Criptografar os dados sensíveis armazenados no Data Lake.
- Chaves de API devem ser armazenadas em um cofre de senhas.

4.5 Implementar Data Quality

- com ferramentas como Great Expectations

5. Considerações Finais

A solução desenvolvida apresenta um pipeline robusto para ingestão, processamento e armazenamento de dados em tempo real, com suporte a monitoramento e observabilidade. As tecnologias utilizadas garantem escalabilidade e flexibilidade, atendendo às demandas do case.

Com as melhorias sugeridas, a solução pode ser ainda mais eficiente e confiável, garantindo maior governança e capacidade de lidar com volumes crescentes de dados.